

1. What this project does

This project builds a UART transmitter (UART TX) in Verilog. It takes an 8-bit byte of data and sends it out one bit at a time on a single wire (**tx_output**), following the standard UART frame format: a start bit (0), then 8 data bits, then an optional parity bit, then a stop bit (1). The design is split into 4 small building blocks (submodules), which are then connected together inside one top module called **uart_tx_top**.

Ports (from the project specification):

Signal	Direction	Meaning
clk	input	Clock – everything updates on its rising edge
rst	input	Asynchronous, active-low reset (0 = reset)
p_input	input	8-bit data byte to send
v_input	input	1-clock pulse: "new data is ready, start sending"
p_en	input	0 = no parity bit, 1 = include a parity bit
p_bit	input	0 = even parity, 1 = odd parity
tx_output	output	The serial line – this is what actually gets sent
busy	output	High while a frame is being transmitted

2. The 4 submodules

Controller

This is the brain of the design. It is a Finite State Machine (FSM) with 5 states: IDLE, START, DATA, PARITY, STOP. It does not touch tx_output directly – it only decides which phase of the transmission we are in, and passes that information (state, bit_index, load) to the other modules. When v_input pulses high in IDLE, it moves to START, raises busy, and pulses load for one clock so the Serializer grabs the incoming byte. It then walks through DATA (counting 8 bits), optionally PARITY, then STOP, and back to IDLE.

Serializer

Holds the 8-bit byte in an internal register (data_reg). When the controller pulses load, it copies p_input into data_reg and keeps that frozen copy even if p_input changes later. Every clock, it outputs data_reg[bit_index] – the controller's bit_index counter picks which of the 8 bits to expose right now.

Parity_calc

Pure combinational logic (no clock). It XORs all 8 bits of the input together (this is called a reduction XOR, written as ^pin in Verilog) to get the even parity value. If p_bit is 1 (odd parity requested), it simply inverts that result.

Mux_tx

The output selector. Based on the current state coming from the Controller, it decides what actually drives tx_output: idle high (1), the start bit (0), the current data bit, the parity bit, or the stop bit (1). This is a simple case statement acting as a multiplexer.

uart_tx_top

This module has no logic of its own – its only job is wiring the 4 submodules above together, so that from the outside it behaves like one single block matching the port table on page 1. This is what satisfies the requirement of having "modules containing submodules."

Testbench (uart_tx_top_tb)

This is not part of the real design – it exists only for simulation. It generates a clock (toggling every 5 time units), applies reset, then feeds three test cases into the design one after another: (1) no parity, (2) even parity, (3) odd parity. It then stops the simulation so the waveform can be inspected.

3. How to choose the parity type

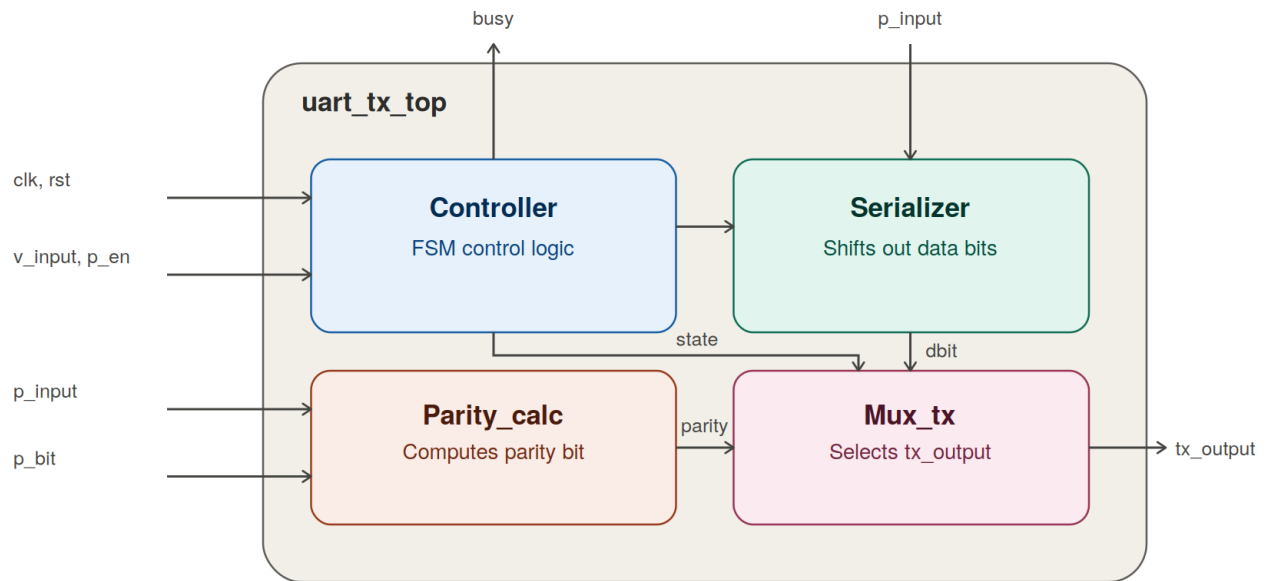
Parity is controlled by two signals together, p_en and p_bit:

Wanted	p_en	p_bit
No parity	0	doesn't matter
Even parity	1	0
Odd parity	1	1

The testbench used for this project already runs all three cases automatically, one after another, in a single simulation run: first with p_en=0 (no parity), then p_en=1 & p_bit=0 (even parity), then p_en=1 & p_bit=1 (odd parity).

4. Module structure diagram

This diagram shows how `uart_tx_top` wires the 4 submodules together, and where each port from the specification connects.



Controller receives `clk`, `rst`, `v_input` and `p_en`, and drives `state` and `bit_index` into the `Serializer` and `Mux_tx`, plus the `busy` output directly. `Serializer` takes `p_input` and outputs `dbit`. `Parity_calc` takes `p_input` and `p_bit` and outputs `parity`. Both `dbit` and `parity` feed into `Mux_tx`, which selects one of them (or a fixed 0/1) to drive `tx_output`, based on the current state.

5. Full Verilog code

controller.v

```
module controller(clk,rst,v_input,p_en,busy,state,bit_index,load);
input clk;
input rst;
input v_input;
input p_en;
output reg busy;
output reg [2:0] state;
output reg [3:0] bit_index;
output reg load;
localparam IDLE=0;
localparam START=1;
localparam DATA=2;
localparam PARITY=3;
localparam STOP=4;
always @(posedge clk or negedge rst) begin
if(rst==0) begin
state<=IDLE;
busy<=0;
bit_index<=0;
load<=0;
end
else begin
case(state)
IDLE: begin
load<=0;
if(v_input==1) begin
state<=START;
busy<=1;
load<=1;
end
end
START: begin
load<=0;
bit_index<=0;
state<=DATA;
end
DATA: begin
if(bit_index<7)
bit_index<=bit_index+1;
else begin
bit_index<=0;
if(p_en==1)
state<=PARITY;
else
state<=STOP;
end
end
PARITY: begin
state<=STOP;
end
STOP: begin
state<=IDLE;
busy<=0;
end
default: state<=IDLE;
endcase
end
end
endmodule
```

serializer.v

```
module serializer(clk,rst,load,pin,bit_index,dbit);
input clk;
input rst;
input load;
input [7:0] pin;
input [3:0] bit_index;
output dbit;
reg [7:0] data_reg;
always @(posedge clk or negedge rst) begin
if(rst==0)
data_reg<=0;
else if(load==1)
data_reg<=pin;
end
assign dbit=data_reg[bit_index];
endmodule
```

parity_calc.v

```
module parity_calc(pin,p_bit,parity);
input [7:0] pin;
input p_bit;
output parity;
assign parity=(p_bit==0)?(^pin):(~(^pin));
endmodule
```

mux_tx.v

```
module mux_tx(state,dbit,parity,tx_output);
input [2:0] state;
input dbit;
input parity;
output reg tx_output;
always @(*) begin
case(state)
0: tx_output=1;
1: tx_output=0;
2: tx_output=dbit;
3: tx_output=parity;
4: tx_output=1;
default: tx_output=1;
endcase
end
endmodule
```

uart_tx_top.v

```
module uart_tx_top(clk,rst,p_input,v_input,p_en,p_bit,tx_output,busy);
input clk;
input rst;
input [7:0] p_input;
input v_input;
input p_en;
input p_bit;
output tx_output;
output busy;
wire [2:0] state;
```

```

wire [3:0] bit_index;
wire load;
wire dbit;
wire parity;
controller CTRL(.clk(clk),.rst(rst),.v_input(v_input),.p_en(p_en),.busy(busy),.state(state),.bit_index(bit_index),.load(load),.dbit(dbit),.parity(parity));
serializer SER(.clk(clk),.rst(rst),.load(load),.pin(p_input),.bit_index(bit_index),.dbit(dbit));
parity_calc PAR(.pin(p_input),.p_bit(p_bit),.parity(parity));
mux_tx MUXTX(.state(state),.dbit(dbit),.parity(parity),.tx_output(tx_output));
endmodule

```

uart_tx_top_tb.v

```

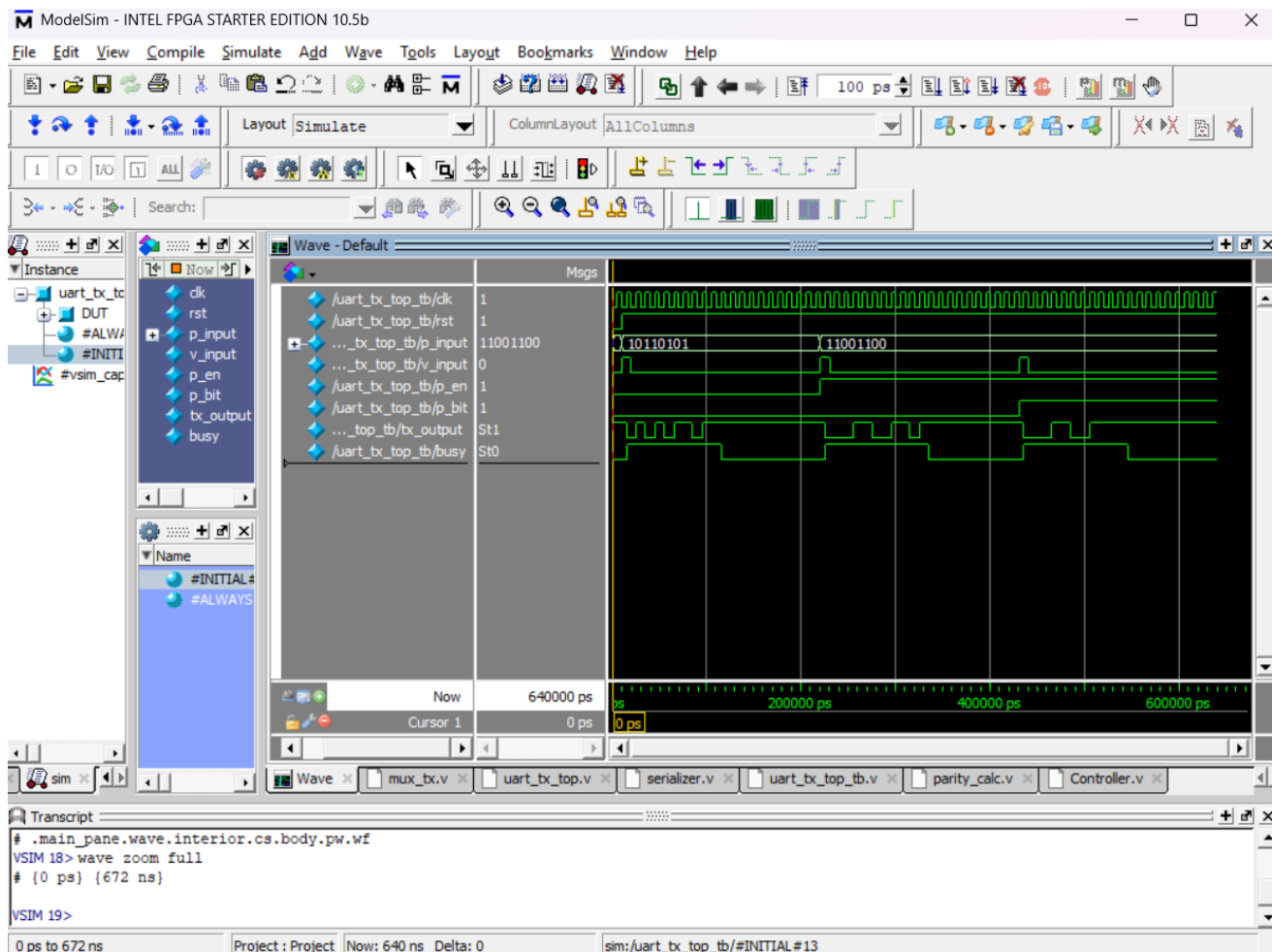
`timescale 1ns/1ps
module uart_tx_top_tb;
reg clk;
reg rst;
reg [7:0] p_input;
reg v_input;
reg p_en;
reg p_bit;
wire tx_output;
wire busy;
uart_tx_top DUT(.clk(clk),.rst(rst),.p_input(p_input),.v_input(v_input),.p_en(p_en),.p_bit(p_bit),.tx_output(tx_output),.busy(busy));
always #5 clk=~clk;
initial begin
clk=0;
rst=0;
p_input=0;
v_input=0;
p_en=0;
p_bit=0;
#10;
rst=1;
p_input=8'b10110101;
p_en=0;
v_input=1;
#10;
v_input=0;
#200;
p_input=8'b11001100;
p_en=1;
p_bit=0;
v_input=1;
#10;
v_input=0;
#200;
p_input=8'b11001100;
p_en=1;
p_bit=1;
v_input=1;
#10;
v_input=0;
#200;
$stop;
end
endmodule

```

6. Simulation waveform

The screenshot below is the full simulation run (0 to 672 ns), zoomed out to show all three test cases in one view. Three separate busy pulses are visible – each one is one complete UART frame:

- Frame 1: p_input = 10110101, p_en = 0 → no parity bit sent
- Frame 2: p_input = 11001100, p_en = 1, p_bit = 0 → even parity
- Frame 3: p_input = 11001100, p_en = 1, p_bit = 1 → odd parity



In each frame, tx_output starts high (idle), drops to 0 for one clock (start bit), then shows the 8 data bits, then (for frames 2 and 3) the parity bit, then returns to 1 for one clock (stop bit) before going back to idle. busy stays high for the entire duration of each frame and drops to 0 once the stop bit finishes.